Integration Testing Report - StreetRace Manager

Roll Number: 2024101145
March 2026

1 Call Graph

The system's integration Call Graphs were hand-drawn and are stored in the integration/diagrams/ directory. These graphs illustrate the flow of control and data between modules such as Registration, Inventory, and the extra modules.

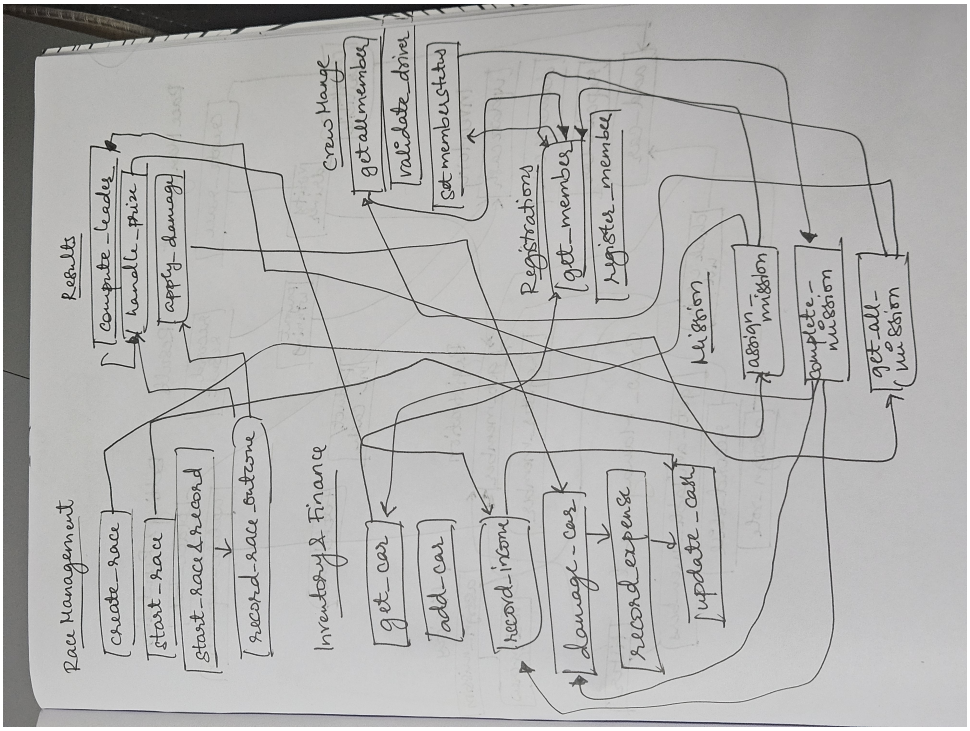

Figure 1: Call Graph

2 Module Descriptions

The system consists of the following core modules located in the racing/ directory. Note: The code module has been renamed to racing to avoid common namespace conflicts during testing.

- **Registration**: Handles crew onboarding and member verification.
- **Crew Management**: Assigns driver/mechanic roles to members.
- **Inventory**: Tracks cars, cash, fuel, and vehicle damage state.
- **Race Management**: Handles race creation, participant logic, and entry checks.
- **Results**: Processes rankings, calculates prize money, and updates financial state.
- **Mission Planning**: Handles role-based heist/mission assignments and resource checks.

3 Extra Modules

1. **Betting Module**: Integrates with Inventory to allow wagering on race results. It deducts stakes and credits winners based on results fetched from RaceManagement.

2. **Reputation Module**: Integrates with Registration to track cumulative experience/rep points across multiple races and missions.

4 Integration Finding Summary

An integration test suite consisting of 10 test cases following the required format was developed to verify the StreetRace Manager's component interaction.

The findings indicate that the multi-module interactions across different components work correctly. For example:

1. **Mission Assignment**: The MissionPlanningModule correctly verifies crew members and their roles.

2. **Race Prize Money**: The ResultsModule correctly processes race rankings and awards prize money, with the InventoryModule showing appropriate balance updates.

5 Integration Test Cases

The integration suite contains 10+ interaction scenarios designed to verify data consistency across the StreetRace modules.

1. **Full Race Lifecycle**
   - **Components**: Registration, Inventory, Race Management, Results.
   - **Input/Flow**: Register "Dom", buy "Charger" (2000), win race (1000 prize).
   - **Outcome**: Cash balance = 4000 (5000 - 2000 + 1000).
   - **Justification**: Verifies the end-to-end race flow and financial ledger consistency across 4 modules.

2. **Mission Safety Check (Mechanic Role)**
   - **Components**: Registration, Inventory, Mission Planning.
   - **Input/Flow**: Damage "Supra", attempt mission without a mechanic.
   - **Outcome**: Mission failure (success == False).
   - **Justification**: Ensures the Mission module queries the Registration module for necessary human capital skills.

3. **Role-Based Race Restriction**
   - **Components**: Registration, Race Management.
   - **Input/Flow**: Register "Mia" as strategist, attempt to race.
   - **Outcome**: success == False (Invalid role error).
   - **Justification**: Validates that business rules in the Race module are correctly enforced via the Registration service.

4. **Financial Guardrail Integration**

- **Components**: Inventory.
- **Input/Flow**: Try to buy "ExpensiveCar" (10000) with 5000 budget.
- **Outcome**: success == False (Insufficient funds).
- **Justification**: Ensures the Inventory module self-regulates its internal state during large transactions.

5. **Registration Deduplication**
   - **Components**: Registration.
   - **Input/Flow**: Register the same name twice.
   - **Outcome**: success == False (Already registered).
   - **Justification**: Ensures data integrity and prevents ID collisions in the onboarding module.
6. **Inventory Asset Destruction**
   - **Components**: Inventory, Mission Planning.
   - **Input/Flow**: Delete a car from the inventory, then try to assign it to a mission.
   - **Outcome**: success == False (Car not found in inventory).
   - **Justification**: Validates that Mission Planning relies on real-time Inventory status.
7. **Mission Profit Synchronization**
   - **Components**: Mission Planning, Results, Inventory.
   - **Input/Flow**: Complete heist mission and record 5000 profit.
   - **Outcome**: Total cash balance increases to 10000.
   - **Justification**: Verifies that high-stakes payouts are correctly funneled back into the primary ledger.
8. **Crew Management Scalability**
   - **Components**: Registration, Crew Management.
   - **Input/Flow**: Bulk register 10 drivers.
   - **Outcome**: Crew manager returns 10 available members.
   - **Justification**: Ensures the cross-registration bridge between modules doesn't scale poorly or lose records.
9. **Multi-Role Crew Requirement**
   - **Components**: Mission Planning, Registration.
   - **Input/Flow**: Mission requiring both a driver and a mechanic for a damaged car.
   - **Outcome**: Mission failure if only drivers are assigned.
   - **Justification**: Checks boolean logic for complex multi-participant resource checks.
10. **Future Transaction Bridge**
    - **Components**: Results, Inventory.
    - **Input/Flow**: Use prize money from Race 1 to buy a high-value car in a future step.
    - **Outcome**: Transaction succeeds only after a previous win.
    - **Justification**: Validates chronological state progression across different module interactions.